

IFS-10 File Copy Integration System

Deployment & Configuration Documentation

Document Version	1.0
Date	April 2026
Application	IFS-10 File Copy Service
Environment	Pre-Production & Production
Trigger	Cron — 08:15, 16:15, 00:15 (every 8 hours)
Classification	Internal

1. Overview

The IFS-10 File Copy Integration System is a scheduled Python service that automates the transfer of counter and measurement files between SMB network shares across multiple environments. It copies new files from source servers to destination servers, persists relative humidity (RH) meter readings to a central database, and archives processed files — all orchestrated via an async execution pipeline.

Purpose

This service ensures that IFS counter data files produced by terminal equipment are reliably replicated from source shares to downstream destinations (IFS-AD share and archive). For environments that include RH sensor data, it additionally parses CSV payloads and upserts time-series records into the operational database with shift classification.

Key Features

- Async file transfer using smbclient (asyncio subprocess)
- Multi-environment support: Production, Pre-Production, and Pre-Production Fuel
- Shift-based classification of RH readings (Night / Morning / Afternoon)
- Database upsert of eq_rh records via CTDataBase ORM
- Automatic deduplication — only files absent from archive are processed
- Temporary directory lifecycle management (create → use → cleanup)
- Structured logging via shared myLogger module
- Automatic log rotation — files older than 15 days are deleted

Technology Stack

Component	Technology
Programming Language	Python 3.9+
Async Runtime	asyncio (standard library)

File Transfer Protocol	SMB via smbclient CLI
Database	Microsoft SQL Server (pyodbc)
Scheduling	Cron (08:15 / 16:15 / 00:15)
Logging	Custom myLogger module

2. System Architecture

The application is a single Python script (IF_FPRD_V2.22.10.24) executed on demand by cron. It resolves the correct server configuration based on the environment argument, discovers new files on the source SMB share, copies them to a local temporary directory, optionally writes data to the database, then pushes each file to both the destination and archive shares.

2.1 Entry Point & Execution Flow

The script is invoked three times per cron run — once per environment — from the `__main__` block:

```
asyncio.run(main(env='production'))
asyncio.run(main(env='Pre_Prod'))
asyncio.run(main(env='Pre_Prod_fuel'))
```

Each invocation follows this pipeline:

- Initialize the temporary working directory
- Resolve the current month file pattern (YYYYMM or YYYY-MM for fuel)
- List files on the source share matching the pattern
- List files already present in the archive share
- For each source file not in the archive: copy to temp → write to DB (non-fuel) → push to destination and archive
- Cleanup the temporary directory (always runs, even on failure)

2.2 File Discovery Module

The `get_server_files()` function uses `smbclient dir` to list files on a remote SMB share and filters by the current month pattern. Returns a list of matching filenames.

2.3 Async File Copy

The `copy_to_temp()` coroutine uses `asyncio.create_subprocess_shell` to invoke `smbclient` get non-blocking. This allows future parallelism without blocking the event loop. The synchronous `copy_to_destination()` uses `subprocess.run` for destination writes.

2.4 Database Write Module (`write_2_db`)

Applicable to Production and Pre-Production environments only (not `Pre_Prod_fuel`). For each transferred CSV file, the module:

- Reads and parses the CSV with `DictReader` (semicolon delimiter)
- Filters rows where `ParameterCode == 'RH'`
- Parses `RegDate` with format `dd/MM/yyyy HH:mm`
- Classifies each reading into a shift: Night (00:00–07:59), Morning (08:00–15:59), Afternoon (16:00–23:59)

- Upserts all filtered rows into the eq_rh database table via CTDDataBase.insert_or_update()

3. Environment Configuration

Three environments are defined in the server_config dictionary within main(). Each environment specifies independent source, destination, archive shares, credentials, and temp directory.

Parameter	Production	Pre-Production	Pre-Prod Fuel
src_server	//10.212.12.180/ifs_10_counters_prod/	//10.212.12.180/ifs_10_counters_test	//10.212.12.97/ifs-preprod
dst_server	//10.212.12.245/ifs-ad/	//10.212.12.245/ifs-ad2	//10.212.12.240/cms2
dst_archive	//10.212.12.245/Archive/	//10.212.12.245/Archive2	//10.212.12.240/archive2
dst_password	*****	*****	*****
File Pattern	YYYYMM	YYYYMM	YYYY-MM
DB Write	Yes (eq_rh)	Yes (eq_rh)	No

4. Scheduling

The script is triggered by a cron job three times per day, aligned with the three operational shifts:

Trigger Time	Shift Covered	Cron Expression
08:15	Morning (08:00–15:59)	15 8 * * *
16:15	Afternoon (16:00–23:59)	15 16 * * *
00:15	Night (00:00–07:59)	15 0 * * *

Recommended Crontab Entry

```
15 8,16,0 * * * cd /home/admin/Projects/IFS10_invoice && /usr/bin/python3 IF_FPRD.py >>
log/app.log 2>&1
```

5. Deployment Steps

5.1 Pre-requisites

Item	Requirement	Notes
Operating System	Linux (Ubuntu 24.04 LTS)	Host: project server

Python Version	Python 3.8+	With asyncio support
smbclient	samba-client package	sudo apt install samba-client
Database Access	SQL Server (pyodbc)	ODBC Driver 17 for SQL Server
Network Access	SMB (TCP 445) to all server IPs	10.212.12.180 / .245 / .97 / .240
User Permissions	admin	Write access to project and temp directories
Disk Space	Minimum 5 GB	Logs + temp files

5.2 Required Python Packages

Install via pip:

```
pip install pyodbc
```

Standard library modules used (no install required): asyncio, csv, json, os, shutil, datetime, time, timedelta, sys, traceback, subprocess

5.3 Deployment Process

- Create project directory: `sudo mkdir -p /home/admin/Projects/IFS10_invoice`
- Deploy application files: IF_FPRD.py, db.py, myLogger.py
- Create log directory: `mkdir -p log`
- Create temp directory: `mkdir -p Temp2`
- Test SMB connectivity to all three source and destination shares (see Section 5.4)
- Test database connectivity (see Section 5.5)
- Run manually: `python3 IF_FPRD.py`
- Configure cron entry (see Section 4)

5.4 Test SMB Connectivity

```
smbclient //10.212.12.180/ifs_10_counters_prod/ -U Administrator%***** -c 'dir'
smbclient //10.212.12.245/ifs-ad/ -U Administrator%***** -c 'dir'
```

5.5 Test Database Connectivity

```
python3 -c "import pyodbc; conn = pyodbc.connect('DRIVER={ODBC Driver 17 for SQL Server};SERVER=<DB_HOST>;DATABASE=<DB>;UID=<USER>;PWD=<PASS>'); print('OK')"
```

6. File Structure

```
/home/admin/Projects/IFS10_invoice/
├─ IF_FPRD.py          # Main script - file copy, DB write, scheduling
├─ db.py              # CTDataBase ORM - insert_or_update()
├─ myLogger.py        # Shared logging module
├─ Temp2/             # Temporary working directory (auto-created/cleaned)
└─ log/               # Application logs (auto-rotated after 15 days)
```

7. Troubleshooting

Issue	Likely Cause	Solution
SMB connection failure	Wrong credentials or firewall	Test smbclient manually. Verify TCP 445 is open to target IPs.
No files found on source	Incorrect file pattern or share path	Verify current month pattern (YYYYMM). Check share path in server_config.
DB write fails	ODBC connection error or schema mismatch	Check DB credentials in db.py/.env. Verify eq_rh table schema.
Files not reaching destination	File already in archive (dedup skip)	Check archive share. If re-push needed, remove file from archive.
Temp directory not cleaned	Script crashed before finally block	Manually delete /home/admin/Projects/IFS10_invoice/Temp2/*
Permission denied on SMB put	Wrong destination credentials	Verify dst_server password in server_config for each env.

Diagnostic Commands

```
# View latest logs
tail -100 /home/admin/Projects/IFS10_invoice/log/app.log

# Run manually for a single environment
cd /home/admin/Projects/IFS10_invoice && python3 -c "import asyncio; from IF_FPRD import main; asyncio.run(main(env='production'))"

# List files on source share
smbclient //10.212.12.180/ifs_10_counters_prod/ -U Administrator% '$*****', -c 'dir'

# Check cron log
grep IF_FPRD /var/log/syslog | tail -30
```

8. Monitoring Checklist

- Monitor log/app.log for ERROR or CRITICAL messages after each cron trigger
- Verify that out_files count is non-zero during active counter periods
- Confirm DB row counts in eq_rh increase each shift after Production / Pre-Prod runs
- Check temp directory is empty between runs (cleanup must complete successfully)
- Alert if smbclient exits non-zero (copy failure logged as ERROR)
- Verify archive share contains current-month files after each run

Support Contacts

Team	Contact
Application Team	APMT IT Development Team (BIH001)

Database Team	N4 Database Administrators
Network Team	Terminal Network Operations